

Software Engineering.

UNIT - I :-

Introduction to Software Engineering : The evolving role of software, changing nature of software, Legacy SW, Software myths

A Generic View of Process :-

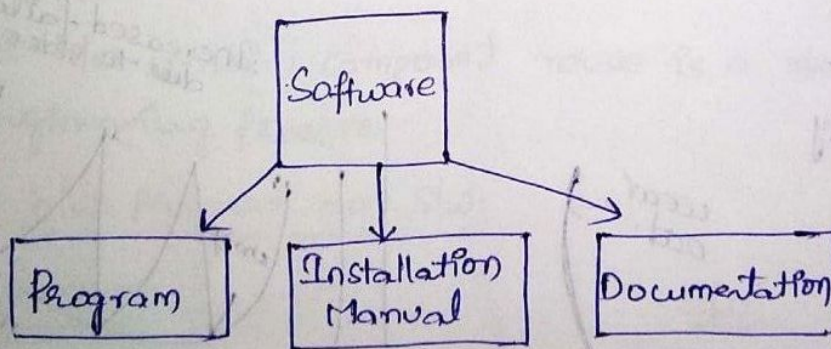
Software Engineering - A Layered technology, A process framework, The Capability Maturity Model Integration (CMMI), process patterns, Process Assessment, personal and team process models.

Process Models :-

The waterfall model, Incremental process models, Evolutionary Process models, specialized process models, The unified process.

Software :-

* Software is a broad term as it includes program, data structure, and documentation which are generated during different stages of life cycle of software development and installation files which are used in a unified manner to perform a particular computing task.



* computer software is the product that software professionals build and then support over the long term.

[Q1] Characteristics of Software :-

[1] Software is developed or engineered; it is not manufactured in classical sense:

- * Although some similarities exist b/w software development and hardware manufacturing, but few activities are fundamentally different.

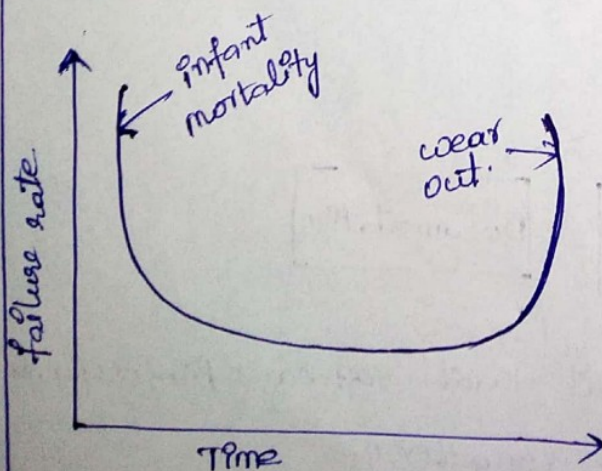
- * In both activities high quality is achieved through good design, but the manufacturing phase for hardware can introduce quality problems than software.

[2] Software doesn't "~~wear out~~" wear out "

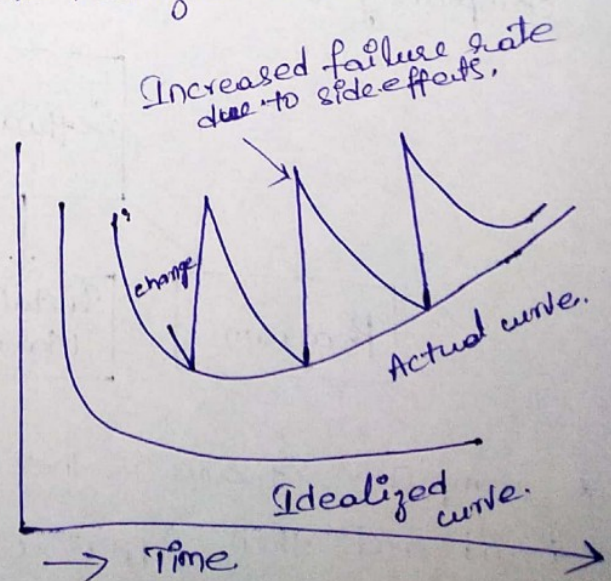
- * H/w components suffer from the growing effects of dust, vibration, abuse, temperature extremes, and many other environmental maladies. Stated simply the hardware begins to wear out.

- * S/w is not susceptible to the environment maladies that cause hardware to "wear out."

In theory, therefore, the failure rate curve for software should take the form of the "idealized curve."



H/w failure curve.



S/w failure curve.

* when a hardware component wears out, it is replaced by a spare part.

* there are no software spare parts.

* Every software failure indicates an error in design or in the process through which design was translated into machine executable code.

* therefore, the slw maintenance tasks are considerably complex than h/w maintenance.

* However, the implication is clear — slw doesn't wear out. But it does deteriorate.

[3] Although the industry is moving toward component-based construction, most slw continues to be custom built.

* A slw component should be designed and implemented so that it can be reused in many different programs.

* Modern reusable components encapsulate both data and the processing that is applied to the data, enabling the slw engineer to create new applications from reusable parts.

* In the h/w world, component reuse is a natural part of the engineering process.

Difference b/w program and slw:

Program

- Small in size
- Author himself is user-software
- Single developer
- Ad-hoc development
- Lack proper interface
- Lack proper documentation

Slw

- Large in size.
- Large number.
- Team developer.
- Systematic development
- well defined interface
- well documented.

[Q2] Evolving role of Software.?

Nowadays, software plays a major role with dual activity. It is a product and a vehicle for delivering a product.

- * As a product, it delivers the computing potential embodied by computer hardware or a network of computers that are accessible by local hardware.
- * whether the product or software resides within a mobile phone or operates inside a mainframe computer.
- * Software is an information transformer likely to produce, manage, acquire, modify, display or transmit the information.

Dual role of SW

- * A product
 - Information transformer - producing, managing and displaying.
- * Vehicle for delivering a product.
 - control of computer (operating system), the communication of information (networks) and the creation of other programs.

The Software :-

- * Provides good product with useful information.
- * transforms the data so that it can be more useful in a local context
- * manages business information to enhance competitiveness
- * Provides a gateway to world wide networks like internet

The role of computer software has undergone significant changes over a time span of little more than 50 years.

- * The software has seen many changes since its inception. After all, it has evolved over the period of time against all odds and adverse circumstances.
- * Computer Industry has also progressed at a break-neck speed through the computer revolution, and recently, the network revolution triggered and/or accelerated by the explosive spread of the Internet and most recently the web.
- * Computer Industry has been delivering exponential improvement in price performance, but the problems with software have not been decreasing.
- * Software still come late, exceed budget and are full of residual faults. As per the latest IBM report,
 - " 31% of the projects get cancelled before they are completed,
 - 53% over-run their cost estimates by an average of 189% and for every 100 projects, there are 94 restarts."

Definition of Software Engineering.

- * software Engineering is defined as systematic, disciplined and quantifiable approach for the development, operation and maintenance of software.

—X—

[Q3] The changing Nature of Software?

- * The Software going to develop in different areas depends on the user. So, the developers of SW, would need to know different categories of softwares before they are going to develop the project.
- * There are seven broad categories of computer software present for different users depending on the requirements they requested.

* The 7 broad categories of SW are.

① System Software:-

- * System SW is a collection of programs written to service other programs.
- * The system ~~the~~ SW is characterized by
 - heavy interaction with computer HW.
 - heavy usage by multiple users.
 - concurrent operation that requires scheduling resource sharing and sophisticated process management.

Eg: operating system, compiler,

② Application Software:-

- * Application SW consists of standalone programs that solve a specific business need.
- * Application facilitates business operations or management/technical decision making.
- * It is used to control business functions in real-time.

Eg: MS-office, Adobe reader, photoshop.

③ Engineering & Scientific SW:-

- * This software facilitates engineering functions & tasks.

4
Eg: CAD - computer aided design application which is used to help an engineer in drawing and test the designed information.

④ Embedded SW:-

- * This software resides within a product or system and is used to implement the control features and functions for the end-user and for the system itself.
- * Embedded SW can perform limited functions or significant function and control capability.
- * While designing phase of the manufacturing SW can be embedded inside, once the product is manufactured, the SW can't be changed.

Eg: washing machines, GPS devices, calculator.

⑤ Web Applications:-

- * This kind of SW is developed for users over the internet.
- * Web apps are evolving into sophisticated computing environments that not only provide standalone features, computing functions and content to the end user, but also are integrated with corporate databases and business applications.

⑥ Artificial Intelligence SW:-

- * Artificial Intelligence SW makes use of non numerical algorithms to solve complex problems that are not amenable to computation or straight forward analysis.

Eg: Robotics, expert systems, pattern recognition, artificial neural nets, theorem proving, & game playing.

[Q4] What is Legacy Software?

- * Older programs are often referred to as legacy software.
- * Legacy software systems were developed decades ago and have been continually modified to meet changes in business requirements and computing platforms.
- * Many legacy systems remain supportive to core business functions and are indispensable to the business.
- * Legacy software is characterized by longevity and business criticality.

Changes made to legacy software:-

- * Legacy systems must undergo some significant change. The changes needed are:
 - 1) The SW must be adapted to meet the needs of new computing environments or technology.
 - 2) The SW must be enhanced to implement new business requirements.
 - 3) The SW must be extended to make it interoperable.
 - 4) The SW must be re-architected to make it viable within a new environment.

The quality of legacy SW:-

Legacy systems sometimes have inextensible designs, convoluted code, poor or non-existent documentation, test cases, and results that were never archived.

[Q5] Discuss about Software Myths?

5

- * In Software engineering, myths are the wrong beliefs or misbeliefs about software and the process used to build it.
- * myths are the misleading attitudes that have caused serious problems
- * Software myths propagate false beliefs and confusions in the minds of Managers, users and developers.

3 types of myths.

- 1) Management myths.
- 2) customer myths.
- 3) Developer or Practitioner myths.

[1] Management myths:-

- (a) The members of the organization believe that, they can acquire all the information they want. Standards, procedures, and principles are present to develop the project.

Reality:-

- * Standards are often incomplete, inadaptable and outdated.
- * Developers are often unaware of all the established standards
- * Developers rarely follow all the known standards.

- (b) If the project is behind schedule, increasing the number of programmers can reduce the time gap.

Reality:-

- * Adding more manpower to the project, which is already behind schedule, further delays the project.
- * New workers take longer to learn about the project as compared to those already working on the project.

- (c) If the project is outsourced to a third party, the management can relax and let the other firm develop SW for them.

- * outsourcing SW to a third party doesn't help the organization, which is incompetent in managing and controlling the SW project internally.

[2] Customer Myths:-

* A customer who requests computer SW may be a person, a technical group, the marketing/sales department, or an outside company that has requested SW under contract.

* The common customer myths are

[a] Brief requirements stated in the initial process is enough to start development; detailed requirements can be added later

Reality:-

* Adding requirements at a later stage often repeating the entire development process.

[b] SW is flexible

* Starting development with incomplete and ambiguous requirements often lead to SW failure. Instead, a complete and formal description of requirements is essential before starting development.

[b] Software is flexible; hence SW requirement changes can be added during any phase of the development process.

Reality:- * Incorporating change requests earlier in the development process costs lesser than those that occurs at later stages.

* SW may require redesigning and extra resources.

[3] Developer or practitioner's myths:-

Common developer myths are

[a] SW development is considered complete when the code is delivered.

Reality:-

* 50% to 70% of all the efforts are expended after the SW is delivered to the user.

[b] The success of a SW project depends on the quality of the product produced.

Reality:-

* The quality of programs is not the only factor that makes the project successful. Instead the documentation and SW configuration also plays crucial role.

[C] Software engineering requires unnecessary documentation, which slows down the project.

Reality:-

* Sw engineering is about creating quality at every level of the sw project. Proper documentation enhances quality which results in reducing the amount of rework.

[d] The only product that is delivered after the completion of a project is the working programs.

Reality:-

* The deliverables of a successful project includes not only the working program that also but also the documentation to guide the users for using the sw.

[Q6] Explain the Generic View of Process

* The road map to build high quality sw product is ~~sw~~ Process. Software Process.

* Software process is adopted to meet the need of software engineers and the managers as undertake the development of software product.

* Different types of projects requires different sw processes.

Software Engineering Layers:-

* Sw Engineering is a layered technology.

Definition:-

Software Engineering is the Process of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software.



* Software Engineering is divided into 4 layers.

Fig: Sw Engineering Layers.

(1) The quality Focus:-

- * An engineering approach must rest on quality.
- * The "bed rock" that supports SW Engineering is a Quality Focus.

(2) Process:-

- * The foundation of Software Engineering is the Process Layer.
- * It purely based on technology.
- * Timely development of computer SW
- * Forms the base for management control of SW Projects.

(3) Methods:-

- * Methods provide technical aspects for building SW.
- * Methods encompass a broad array of tasks include communication, requirements analysis, design modeling, Program construction, testing and support.

(4) Tools:-

- * Tools provide automated or semiautomated support for the Process and the methods.
- * When tools are integrated so that information created by one tool can be used by another.
- * A system for the support of SW development is called computer aided SW.

Process Framework:-

- * Process framework is the foundation for a complete SW Process.
- * It has set of small number of framework activities that are applicable to all SW projects, regardless of their size or complexity.
- * A Process framework in turn has set of umbrella activities

that are applicable across the entire software process.

Software Process.

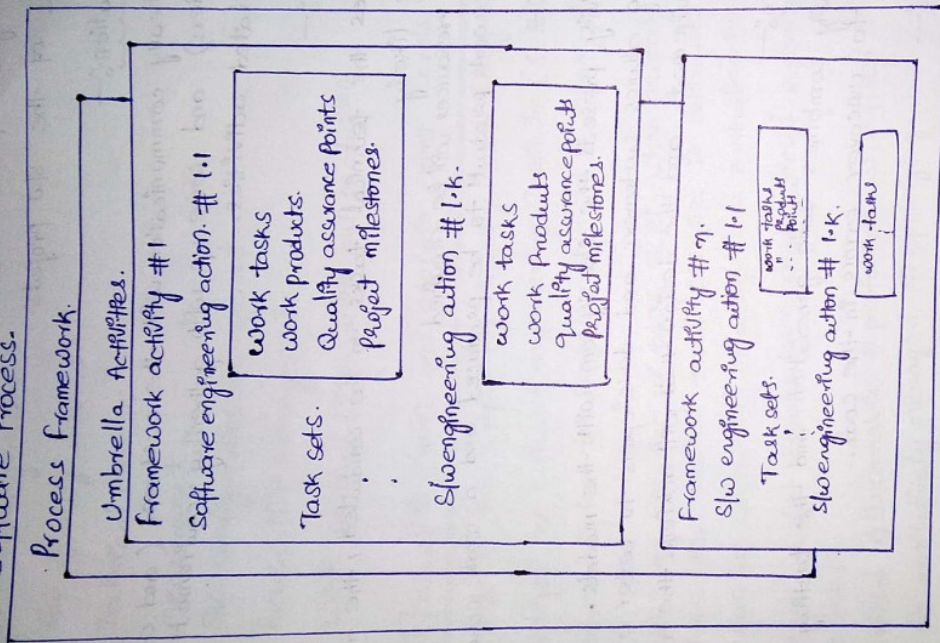


Fig:

* Each framework activity consists of set of software engineering actions

→ A collection of related tasks (called task sets)

work tasks

work products

Quality assurance points

Project milestones.

* Task sets will vary depending on the characteristics of a project

A generic Process Framework :-

A generic process framework is the basis and applicable to the most of the SW projects.

(i) Communication :-

- * Involves heavy communication with the customer (and other stakeholders) and come up with gathering requirements and other related activities.

(ii) planning :-

- * Describes the technical tasks to be conducted, the risks that are likely
 - The resources will be required
 - The work products to be produced and a work schedule.

(iii) Modeling :-

- * This activity presents the SW system with the models.
- * Modeling allows customers and developers to better understand SW requirements and the design that will achieve the required system.

(iv) Construction :-

- * This activity combines code generation and the testing that is required to uncover errors in the code.

(v) Deployment :-

- * Deliver the project to the customer who evaluates the delivered product and provides feedback.

SW engineering Umbrella Activities :-

- * SW engineering process framework activities are followed by a number of umbrella activities.
- * Umbrella activities are applied throughout a SW project and help SW team manage, control progress, quality, change & risk.

activities are

- * Software project tracking and control
- * Risk management
- * Software quality assurance
- * Technical reviews
- * measurement
- * Software configuration management
- * Reusability management
- * work product preparation and production.

[Q7] what is CMMI and explain CMMI Levels.?

* CMMI is the Capability Maturity model Integration which was developed by Software Engineering Institute (SEI) as a process improvement tool for projects, divisions or organizations.

* process as the set of tasks that, when properly performed produces the desired result otherwise it addresses the problems they can be controlled, measured and controlled. Improved.

* CMMI model breakdown organizational maturity into 5 levels.

~~Initial~~ 1) Incomplete 2) performed 3) Defined
4) Quantitatively managed 5) optimizing.

Level 0:- [Incomplete]

* process is not performed or does not achieve all goals defined for

Level 1:- [performed]

* work tasks required to produce required work products are being conducted

Level 2:- [Managed]

* people doing work have access to adequate resources to get job done, stakeholders are actively involved, work tasks and products are monitored, reviewed, and evaluated for conformance

to process description.

Level 3: [Defined]

* Management and engineering process documented, standardized and integrated into organization-wide software process.

Level 4: [Quantitatively Managed]

* Software process and products are quantitatively understood and controlled using detailed measures.

Level 5: [Optimizing]

* Continuous process improvement is enabled by quantitative feedback from the process and testing innovative ideas.

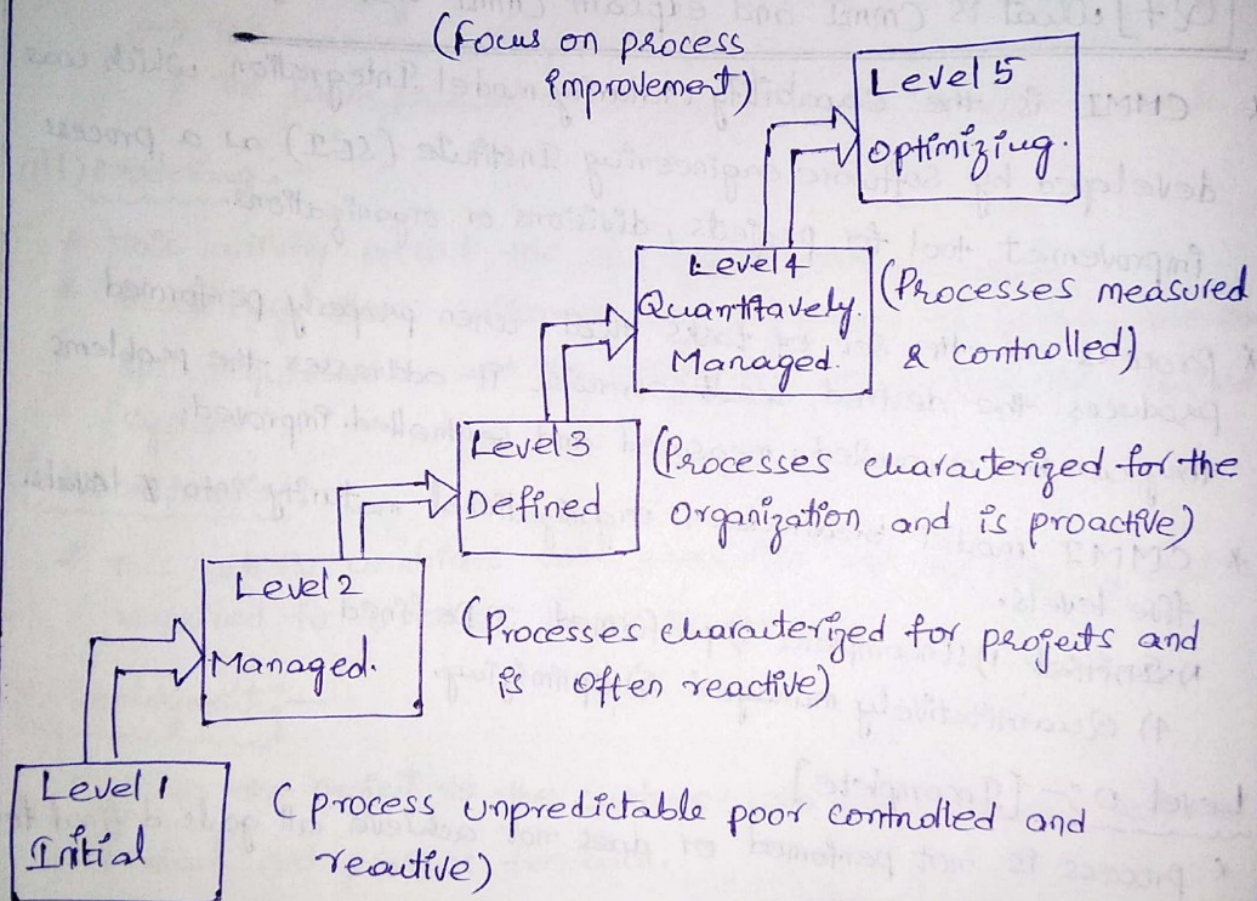
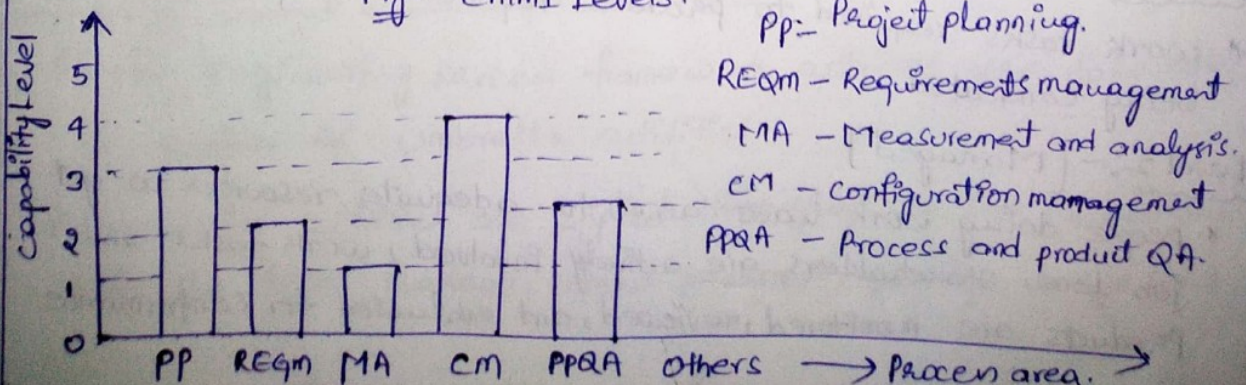
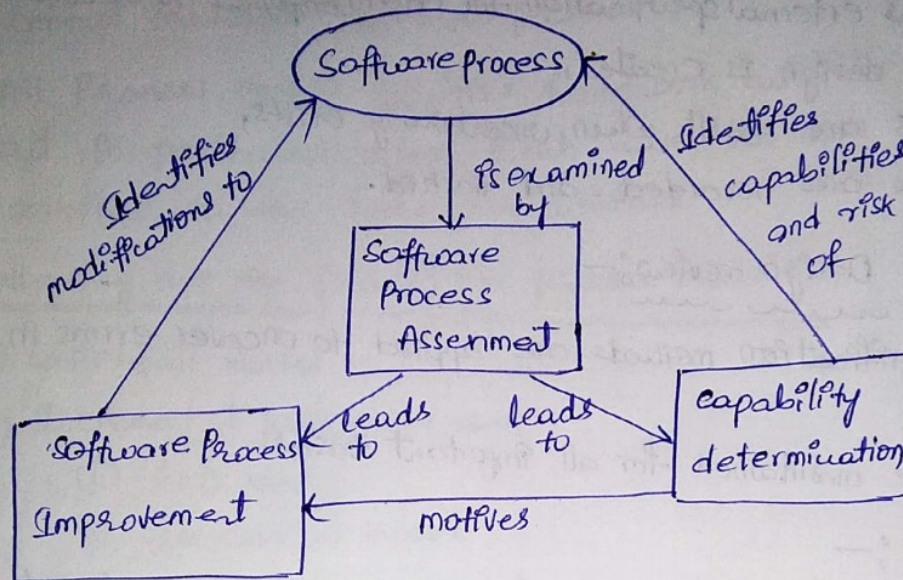


Fig: CMMI levels.



Process Assessment:-

- * The process should be assessed to ensure that it meets a set of basic process criteria that have essential for a successful Software Engineering.
- * This is used by industry professionals.



[Q] Explain personal and Team Process models.?

- * Rsp Personal software process (PSP) and Team software process (TSP), both are largely iterative or evolutionary in the approach to software development.
- * PSP and TSP are interesting but are not pivotal to an understanding of process issues.

Personal software Process (PSP):-

- * The personal software process (PSP) model is good from the perspective that an individual software engineer can use it to improve his or her personal productivity and work product quality.
- * PSP process model defines five activities
 - 1) planning
 - 2) high-level design
 - 3) high-level design review
 - 4) development
 - 5) postmortem.

- * It stresses the need to identify errors early and to understand the types of errors.

1) Planning: It isolates requirements and based these project schedule is created.

2) High-Level Design:—

- * It contains external specification for each component to be constructed, component design is created, prototypes are built when uncertainty exists, all issues are recorded and tracked.

3) High-Level Design review:—

- * Formal verification methods are applied to uncover errors in the design.
- * metrics are maintained for all important tasks.

4) Development:—

- * ~~the component~~ component level design is refined & reviewed
- * code is generated, reviewed, compiled and tested

5) postmortem:—

- * Using the measures and metrics collected, the effectiveness of the process is determined.

Team Software Process:—

- * The goal of TSP is to build a "self-directed" project team that organizes itself to produce high quality sw.
- * Each project is "launched" using a "script" that defines the tasks to be accomplished.
- * Teams are self-directed
- * Measurement is encouraged
- * measures are analyzed with the intent of improving the team process.

Process models.

Prescriptive Process models:-

- * we call the models as prescriptive because they prescribe a set of process elements framework activities, software engineering actions, tasks, work products, quality assurance, and change control mechanism for each product.
- * All Process models specifies framework activities to be carried out in particular order. Each have it's own emphasis workflow to the framework activities.

Following are the prescriptive process models.

- 1) water-fall model
- 2) Incremental process models.
 - (i) RAD model
 - (ii) Incremental model.
- 3) Evolutionary process model
 - (i) prototype model.
 - (ii) spiral model.
 - (iii) concurrent process model.
- 4) Specialized process model.
 - (i) component - Based development
 - (ii) formal methods model.
 - (iii) Aspect - oriented software development
- (5) Unified process.

Q] Explain about waterfall process model ?

- * waterfall model sometimes called the classic life cycle, suggest a systematic, sequential approach to sw development that begins with customer specification of requirements and progresses through planning, modeling, construction and deployment, ~~contin~~
- * There are times when the requirements of a problem are reasonably well understood,
- * when work flows from communication through deployment in a reasonably linear fashion.
- * when well-defined adoptions or enhancements to an existing system must be made.
- * It may also occur in a limited number of new development efforts, but only when requirements are well-defined and reasonably stable.

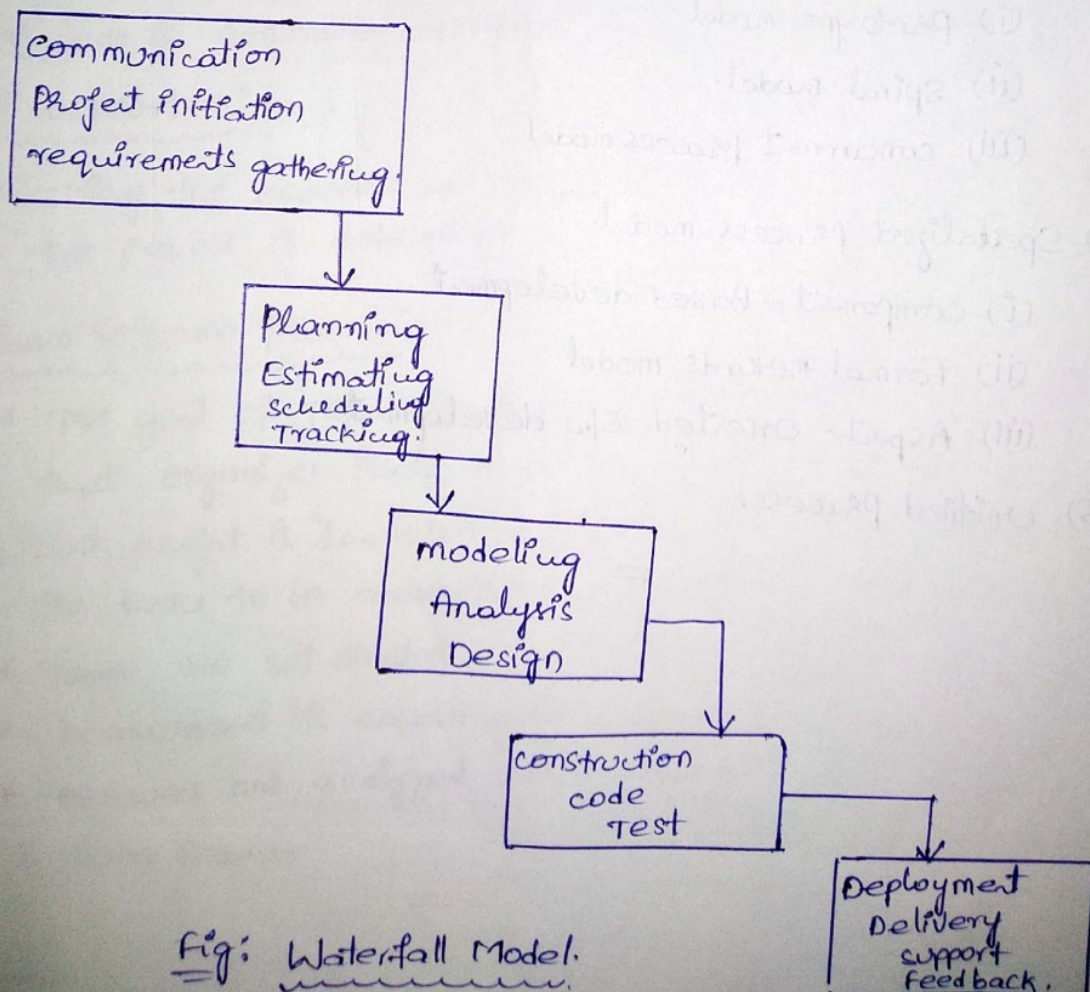


Fig: Waterfall Model.

- * It is a Sequential model, The phases in this model are communication, planning, modeling, construction, and deployment.
- * ~~The input~~ In waterfall model, upon completion of one stage only next stage will start working.

Limitations :-

- * The nature of the requirements will not change during development; during evolution.
- * The model implies that you should attempt to complete a given stage before moving on to the next stage.
- * Does not account for the fact that requirements constantly change.
- * It also means that customers cannot use anything until the entire system is complete.
- * The model implies that once the product is finished, everything else is maintenance.
- * Therefore, this model is only appropriate when the requirements are well-understood and changes will be fairly limited during the design process.

(2) Incremental Process models.

(i) Incremental model :-

- * Incremental model in SW engineering is a one which combines the elements of waterfall model in an iterative manner.
- * It delivers a series of releases called increments which provide progressively more functionality for the client as each increment is delivered.
- * In the incremental model of SW engineering, the waterfall model is repeatedly applied in each increment.

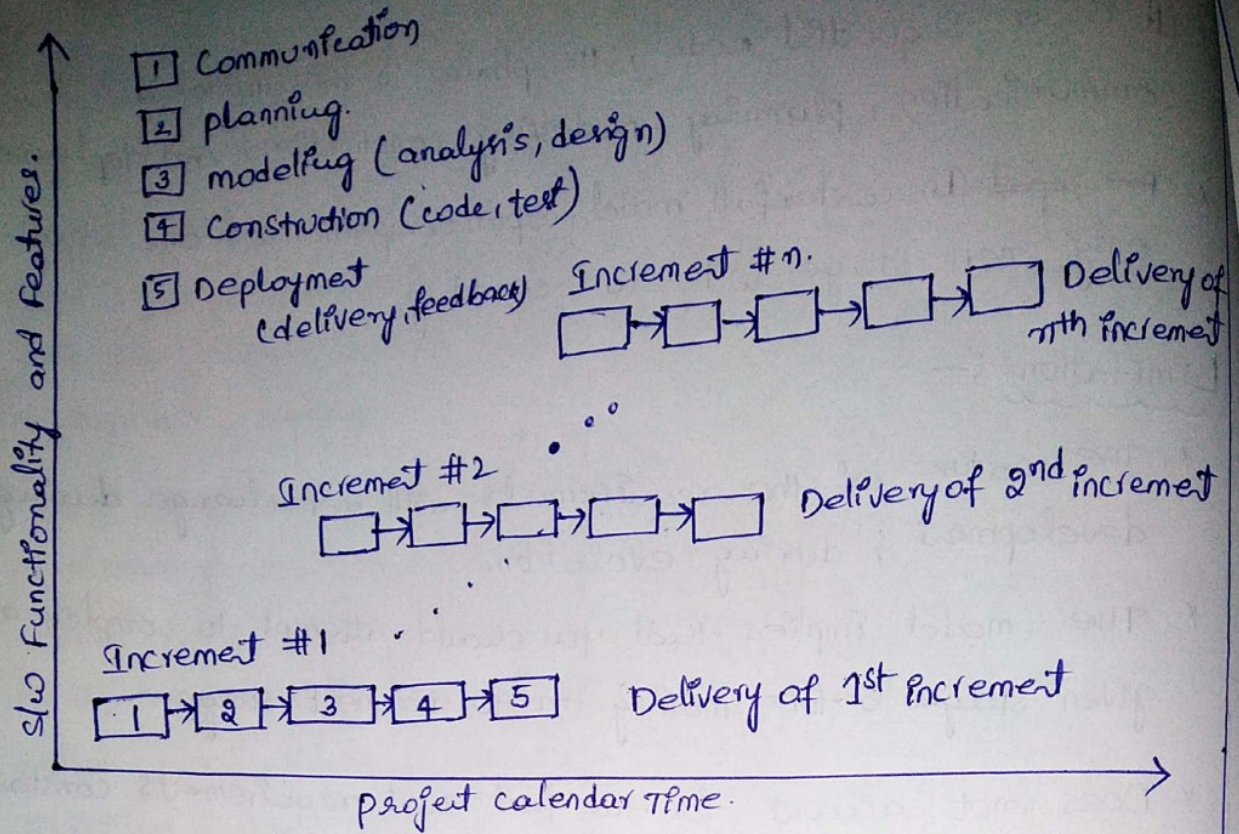


Fig: Incremental model.

As from the diagram, you can see there are 5 phases (tasks) which are carried out in each increment.

- * The first increment is often a core product where the necessary requirements are addressed, and the extra features are added in the next increments.
- * The core product is used and evaluated by the client.
- * Once the customer assesses the core product, there is plan development for the next increment.
- * Thus in every increment, the needs of the client are kept in mind, and more features and functions are added, and the core product is updated.
- * This process continues until the complete product is produced.
- * One of the benefits of the incremental process model is that it can be planned to manage technical risks.

Advantages of Incremental model:-

- * Initial product delivery is faster.
- * Lower initial delivery cost.
- * Risk of changing requirement is reduced.
- * workload is ~~less~~ less.

Disadvantages:-

- * Each phase of an iteration is rigid and do not overlap each other.
- * It requires a good planning designing.
- * Rectifying a problem in one unit requires correction in all the units and consumes lot of time.

(ii) The RAD Model:-

- * Rapid Application Development (RAD) is an incremental software process model that emphasizes a short development cycle.
- * RAD is a "high-speed" adaption of the waterfall model, in which rapid development is achieved by using a component based construction approach.
- * If requirements are well understood and project scope is constrained, the RAD process enables a development team to create a fully functional system within a short period of time.

Drawbacks

- (1) For large, but scalable projects, RAD requires sufficient human resources to create the right number of RAD teams.
- (2) If a system cannot properly be modularized, building the components necessary for RAD will be problematic.

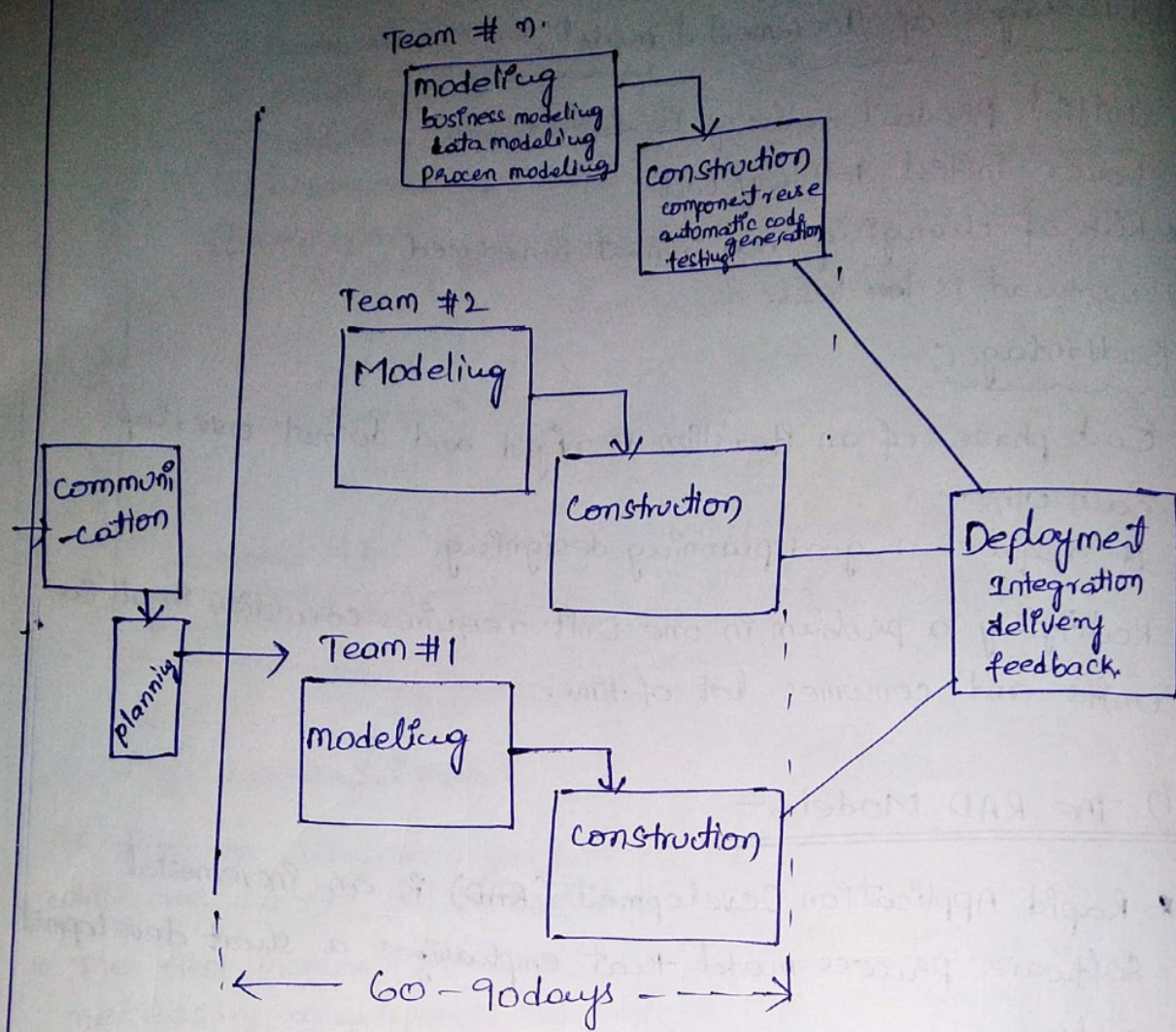


Fig: The RAD model

III. Evolutionary process models.

- * Evolutionary models are iterative type models. They allow to develop more complete versions of the software.
- * Following are the evolutionary process models.
 1. prototyping model
 2. The spiral model.
 3. concurrent development model.

(i) The prototyping model:—

- * The basic idea here is that instead of freezing the requirements before a design or coding can proceed,

13
a throwaway prototype is built to understand the requirements.

* By using this prototype, the client can get an "actual feel" of the system, since the interactions with prototype can enable the client to better understand the requirements of the designed system.

* The prototyping paradigm begins with communication. The s/w engineer and customer meet and define the overall objectives for the s/w, identify whatever requirements are known.

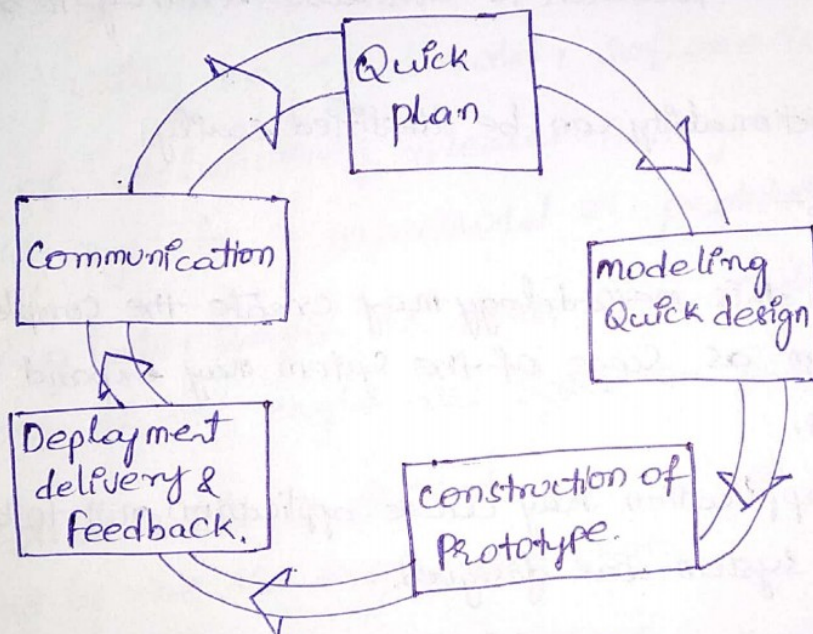


Fig: prototyping model.

*

* A prototyping iteration is planned ~~too~~ quickly and modeling occurs.

* The quick design focuses on a representation of those aspects of the s/w that will be visible to the customer/end user.

* (eg: human interface layout or output display formats)

* The quick design leads to the construction of a prototype.

* The prototype is deployed and then evaluated by the customer/user.

* Feedback is used to refine the requirements for the S/W. Iteration occurs as the prototype is tuned to satisfy the needs of the customer, while at the same time enabling the developer to better understand what needs to be done.

Advantages:-

- * users are actively participated in the development.
- * Errors can be detected much earlier.
- * Quicker user feedback is available leading to better solutions
- * missing functionality can be identified early.

Disadvantages:-

- * Practically, this methodology may create the complexity of the system as scope of the system may expand beyond original plans.
- * Incomplete application may cause application not to be used as the full system was designed.

When to use Prototype model:-

- * prototype model should be used when the desired system needs to have a lot of interaction with the end users.
- * Typically, online systems, web interfaces have a very high amount of interaction with end users, are best suited for Prototype model.

2) The Spiral model :-

14.

* The spiral model is evolutionary process model that couples the iterative nature of prototyping with the controlled and systematic aspects of the waterfall model. It has two distinguishing features

- (a) A cyclic approach for incrementally growing a system's degree of definition and implementation while decreasing its degree of risk.
- (b) A set of anchor point milestones for ensuring stakeholder commitment to feasible and mutually satisfactory solutions.

Using the spiral model, software is developed in series of evolutionary releases. During early stages, the release might be a paper model or prototype.

* During later ~~stages~~ iterations, increasingly more complex versions of the engineered system are produced.

* A spiral model is divided into set of framework activities divided by the s/w engineering team.

* As this evolutionary process begins, the software team performs activities that are implied by circuit around the spiral in clockwise direction, beginning at the center. Risk is considered as ~~revolution~~ each revolution is made.

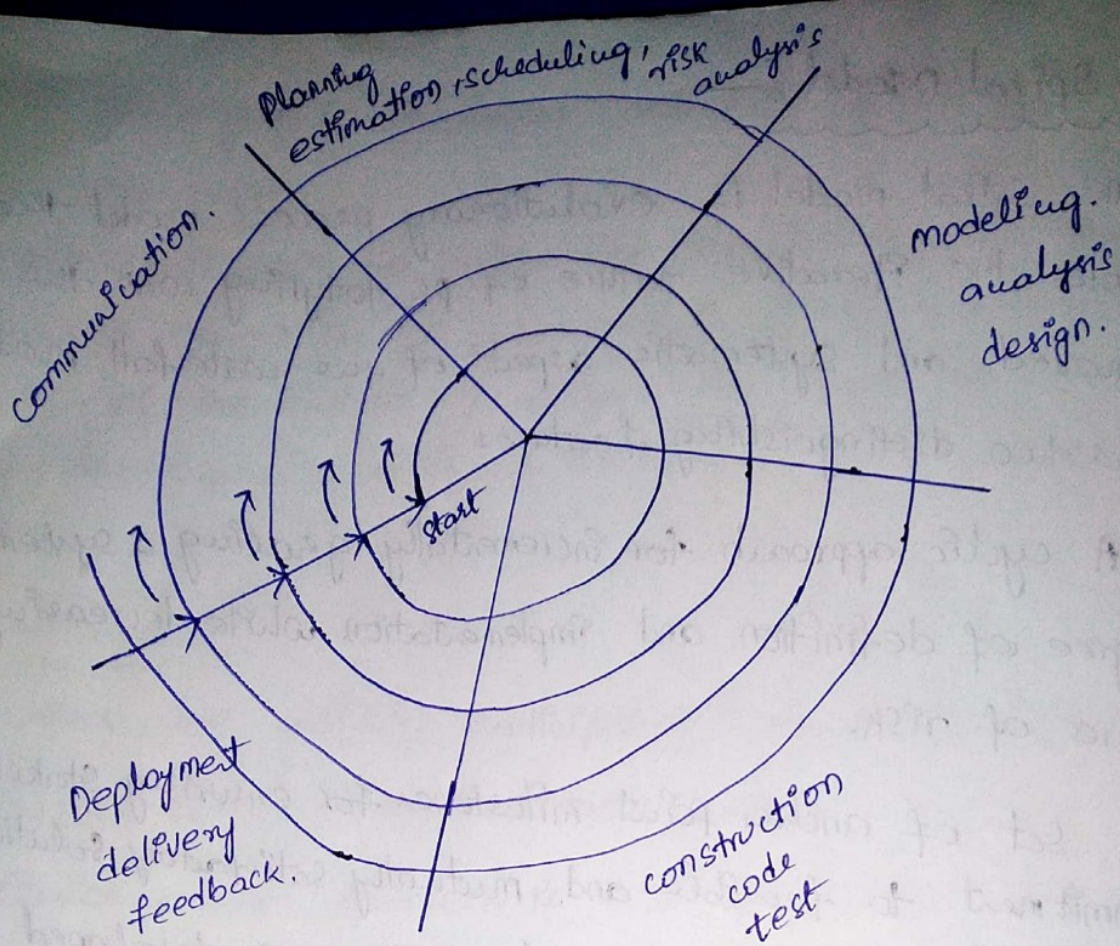


Fig: A typical spiral model.

The first circuit around the spiral might result in the development of a product specification; subsequent passes around the spiral might be used to develop a prototype and then progressively more sophisticated versions of the software.

- * Each pass through the planning region results in adjustments to the project plan.
- * cost and schedule are adjusted based on feedback derived from the customer after delivery.
- * Unlike other process models that end when software is delivered, the spiral model can be adapted to apply throughout the life of the software.

THE CONCURRENT DEVELOPMENT MODEL.

(3)

- * The concurrent develop model, sometimes called concurrent engineering, can be represented schematically as a series of framework activities, software engineering actions of tasks, and their associated states.
- * The concurrent model is more appropriate for system engineering projects where different engineering teams are involved.

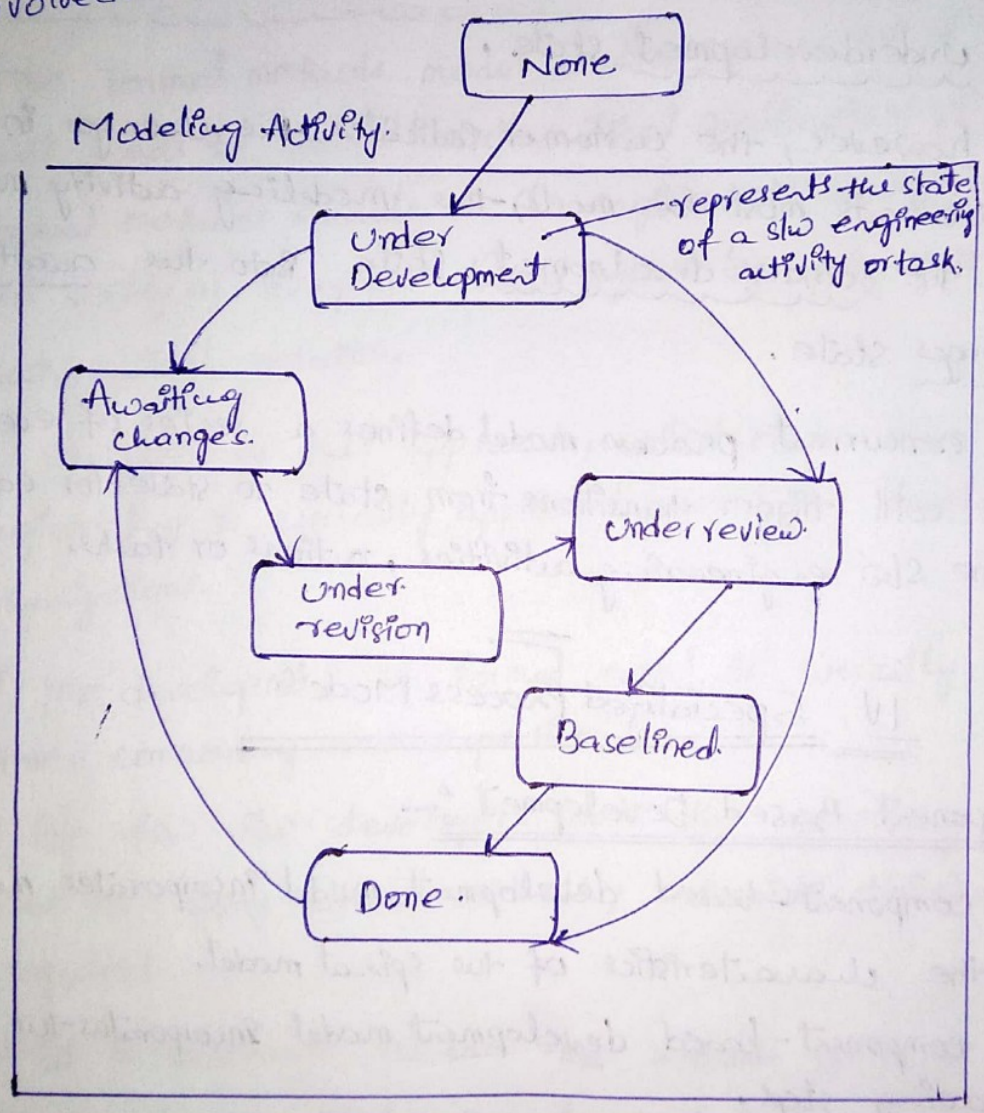


Fig: one Element of the concurrent process model.

- * Figure above provides a schematic representation of one software engineering task within the modeling activity for the concurrent process model.

- * The activity - modeling may be in any one of the states noted at any given time.
- * All activities exist concurrently but reside in different states. For example, early in the project the communication activity has completed its first iteration and exists in the awaiting changes state.
- * The modeling activity which existed in the none state while initial communication was completed now makes a transition into underdevelopment state.
- * If, however, the customer indicates the changes in requirements must be made, the modeling activity moves from the under development state into the awaiting changes state.
- * The concurrent process model defines a series of events that will trigger transitions from state to state for each of the S/W engineering activities, actions or tasks.

IV Specialized Process Models.

(a) Component Based Development :-

- * The component-based development model incorporates many of the characteristics of the spiral model.
- * The component-based development model incorporates the following steps.
 - (i) Available component-based products are researched and evaluated for the application domain.
 - (ii) Component integration issues are considered
 - (iii) S/W architecture is designed to accommodate the components.

- 16
- (iv) components are integrated into the architecture;
 - (v) comprehensive testing is conducted to ~~ens~~ ensure proper functionality.

The component-based development model leads to software reuse, and ~~reus~~ reusability provides software engineers with a number of measurable benefits.

(b) The Formal Methods model :-

- * The formal methods model encompasses a set of activities that leads to formal mathematical specifications of software.
- * Formal methods enable a software engineer to specify, develop and verify a computer-based system by applying a rigorous, mathematical notation.
- * A variation of this approach, called clean-room software engineering is currently applied by some software development organizations.

- (i) The development of formal model is currently quite time-consuming and expensive
 - (ii) B/c few software developers have the necessary background to apply formal methods, extensive training is required.
 - (iii) It is difficult to use the methods as a communication mechanism for technically unsophisticated customers.
-

V. UNIFIED PROCESS MODEL

Unified Process is incremental and iterative process model extensively used for object oriented software development.

had to be
* In this model, requirements or features ~~are~~ identified before hand. And then over several iterative development the sw is developed and each time it is deployed at the client side, client feedback is obtained.

* Unified process development occurs over 4 phases.

1) Inception

2) Elaboration

3) Construction

4) Transition.

* Each of these above phases, consists of many increments.

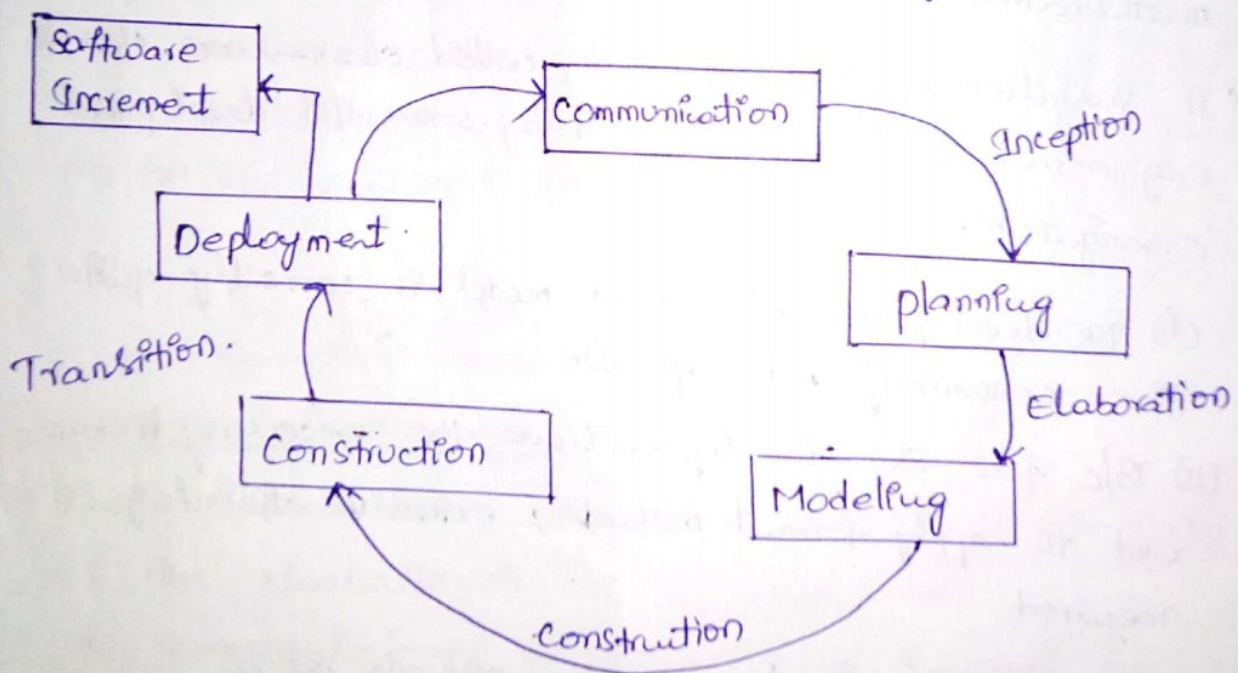


Fig: Unified process for one increment.

* In the Inception phase, the initial ~~use~~ usecase model is proposed, that is the features, the business case, risk list, Project plan, planning documents, and any prototype.

* In the elaboration phase, models are made like analysis model, preliminary model, manual use case model and so on.

* In the construction phase, the slw is developed, the test plan is made, several manuals like user manual and installation manuals are prepared. And finally the developed software increment is deployed at the customer site; The beta test reports and the user feedback is obtained.

Unified process work products.

<u>Inception</u>	<u>Elaboration</u>	<u>Construction</u>	<u>Transition.</u>
* Vision document	* use-case model	* Design model	* slw increment
* Initial use case model	* Requirements Analysis model	* slw components	* Beta test reports
* Initial business case	* Preliminary model	* Test plan	* User feedback.
* initial risk list	* revised risk list	* Test procedure	
* project plan	* preliminary manual	* test cases	
* prototype(s)		* user manual	
		* installation manual	

(Q) SEI-CMMI LEVELS.

- * CMMI is Capability maturity model Integration which is proposed by Software Engineering Institution (SEI)
- * CMMI helps organizations to improve the quality of the slw they developed
- * Adopting to SEI-CMMI model had significant benefits to the business.
- * many commercial organizations adopt CMMI framework for their con improvement initiatives.

* The different levels of SEI-CMMI have been designed so that it is easy for an organization to slowly build its quality system starting from scratch.

* SEI-CMMI classifies 5 maturity levels.

Level 1 : Initial.

Level 2 : Repeatable.

Level 3 : Defined

~~Level~~ Level 4 : managed

Level 5 : optimising.

Level 1 :- [Initial]

- * An organization at this level is characterized by ad hoc activities.
- * Very few or no processes are defined and followed.
- * And no formal project management practices are followed, as a result time pressures at the end of the delivery time, as a result it leads to low quality products.

Level 2 :- [Repeatable]

- * At this level, the basic project management practices such as tracking cost and schedules are established.
- * Configuration management tools are used on items identified for configuration control.
- * Size and cost estimation techniques such as function point analysis, Cocomo models are used.
- * Repeatable level occurs for organizations, when a company produces family of products. Since the products are very similar, the success story on development of one product can be repeated for another.

Level 3 :- [Defined].

- * At this level, the process for both management and development activities are defined and documented.
- * At this level, the organization builds up the capabilities of its employees through periodic training programs. Also review techniques are emphasized and documented.

Level 4 :- [Managed]

- * At this level, the focus is on software metrics. Both process and product metrics are collected.
- * Quantitative quality goals are set for the products and at the time of completion of development it was checked whether the quantitative quality goals for the product are met.

Level 5 :- [Optimising]

- * At this stage process and product metrics are collected, and process measurement data are analyzed for continuous process improvement.
- * At Cmm Level 5, an organization would identify the best software engineering practices and innovations (which may be tools, methods, processes)
- * Level 5 Organization usually have a department whose responsibility is adopting to latest tools and techniques, technologies and propagate them world wide.
- * Level 5 Organizations use configuration management techniques to manage process change, ~~since~~ since process changes occurs continuously.